VIAT & UNIVERSAL WEB

EMBRACE CHAOS WITH **BRUTAL** ORDER

[2 ASYMMETRIC NETWORKS]

[1 UNIFIED NEXUS]

META LAYER

↗HIERARCHICAL PROOF-OF-WORK

>> REAL-TIME & QUANTUM PROOF <<

‖ VIAT.NETWORK | GITHUB | DEVELOPER ‖

NOT BASED ON ANY SDK, BLOCKCHAIN, OR NETWORK
BUILT FOR QUANTUM FROM THE GROUND UP

MADE IN AMERICA - ©2025 TM

VIAT

>> | SUPERSTRUCTURE | HPoW | META LAYER | <<
>> | REALTIME | QUANTUM READY | <<

OVERVIEW

VIAT /ˈviː.æt/ is a Level-1, built-from-scratch transitory post-quantum cryptocurrency built on a Hierarchical Proof-of-Work (H-PoW) architecture designed for real-time decentralized validation, massive scalability, and long-term cryptographic agility. It reimagines blockchain structure by treating every action as its own verifiable block, enabling parallel processing, instant validation, and a compact yet fully immutable history.

Unlike linear blockchains, Viat organizes data through per-wallet chains connected to global audit layers. Wallet chains handle local transaction logic, while auditor networks establish the canonical, verifiable record of truth together forming a decentralized, asynchronous superstructure that scales organically as the network grows.

Initially Viat uses legacy cryptography for speed, on-boarding, and efficiency, while embedding a trapdoor, (a compact post-quantum commitment), in every wallet address. This trapdoor avoids upfront post-quantum storage costs and links each wallet's current keys to *future post-quantum credentials*, enabling the entire network to upgrade automatically. The design ensures a seamless, enforceable, and verifiable transition without any loss of addresses, providing users the best of both worlds as quantum threats emerge.

For the sake of brevity this document primarily focuses on VIAT not the Universal Web.

VIAT NETWORK INFO

SYMBOL: ∀

LAYER: META LAYER

SIGNATURE SCHEMES: [DILITHIUM ◉ ED25519 ◉ SPHINCS+] (QUANTUM READY)

HASH: SHAKE256 (XOF)

KEYED HASH: KMAC256_XOF

GENESIS HASHES: SHAKE256 ▶ [256B - 2048bits] (SHAKE256) >1600bits

KEY EXCHANGE: [KYBER ◉ x25519]

STRICT-HASH: SHA3-256 (32Byte), SHA3-512 (64Byte)

ADDRESS SIZES: [24, 32, 40-64]BYTES (SHAKE256)

PROOF: Hierarchical Proof-of-Work (HPoW)

BLOCKCHAIN TYPE: SUPERSTRUCTURE

BLOCKCHAIN: Hybrid Block lattice [DAG ◉ LINEAR BLOCKCHAIN]

FILESYSTEM: SELF-ORGANIZING FILESYSTEM [VFS]

ENCODING: CBOR ▶ BINARY DATA SERIALIZATION FORMAT

DESIGN VERSION: v5.1.0

DOMAIN SYSTEM: Domain Information System [DIS]

NETWORK PROTOCOL: UW://[UDSP + DIS] {UDP} - (TRANSPORT AGNOSTIC)

QUANTUM: QUANTUM PROOF & QUANTUM READY

THE POST-QUANTUM ERA

INTRODUCTION

Q-DAY

When a sufficiently powerful quantum computer runs Shor's algorithm with enough qubits, it can break all legacy cryptocurrencies overnight (secp256k1, ed25519, BLS). By the time the threat is recognized, it's already too late to transition. Q-Day had gone unnoticed for months or even years, but suddenly the $3.8 trillion blockchain market experiences mass sell-offs, panic, and widespread disruption as the reality sinks in. With no clear solution in sight this is the fate of legacy blockchain networks like Bitcoin, Ethereum, and Solana, utter chaos.

THE PROBLEM

ATTACK SURFACE

Wallets that have exposed legacy public keys allow a quantum attacker to derive private keys directly. Hash based addresses with no transactions are slightly safer, requiring Grover's algorithm to extract the public key first then Shor for the private key.

For some blockchains SHA-256 gives temporary safety for unspent addresses but they remain essentially frozen. Risking a spend for an unspent address risks compromising the public key in flight. If they do try to send they risk their private key being broken the moment the request is sent on the network.

Other networks like SOL use their public key as the address which means they have absolutely no protections as all public keys are exposed on the network meaning all wallets are exploitable by Shor. A major design mistake that lacked post-quantum preparedness.

PRIVATE KEY IMPERSONATION

Shor's algorithm breaks the very assumption that private keys cannot be derived from public keys. Once an attacker can compute the private key, they can generate valid signatures at will and fully impersonate the rightful owner of any address. That level of compromise gives the attacker unrestricted access to funds, permissions, and smart contract authority. In a smart contract environment, this isn't just theft it lets the attacker trigger functions, seize governance roles, bypass multisig, drain treasuries, or rewrite contract-controlled state as if they were every participant at once. The collapse isn't local; it ripples across protocols because the trust model dissolves the moment signature authenticity can no longer be trusted.

IMMUTABILITY A DOUBLE-EDGED SWORD

Centralized systems such as banks can issue systemic upgrades that automatically protect old accounts a quality not present in immutable networks. Blockchain networks like Bitcoin, Ethereum, and SOL can introduce new algorithms or address schemes, existing wallets (secp256k1, ed25519, BLS) remain exposed because they are permanently tied to their public keys. This means their legacy wallets remain indefinitely vulnerable, regardless of future cryptographic network improvements.

IMPOTENT UPGRADES

Even if networks introduce wallet upgrades, many users will never hear about them, others won't bother to migrate, and countless wallets have long been abandoned. Exactly what we have seen already. Mass liquidation of wallets is all but certain. **Existing wallets can't be secured retroactively they will remain broken.**

WALLET FREEZE

For many networks the more likely outcome is a widespread wallet freeze. This means all current Solana, Ethereum, and Bitcoin wallets that exist today will be frozen. Networks may halt entirely to contain exploitation, locking vast amounts of cryptocurrency indefinitely. This response creates a deeper problem: establishing rightful ownership becomes a le-

gal and logistical nightmare. Centralized authorities or committees could emerge to decide which wallets are unfrozen effectively taking control over assets once thought to be trustless. This also changes the nature of those networks transforming them into entities that can effectively “de-bank” users as Governments and or NGOs pressure centralized dev teams to freeze specific wallets. In such a scenario, denying access to a user who still possesses their private keys could be indistinguishable from theft, setting the stage for years of legal disputes, censorship via “de-banking”, and the erosion of decentralization itself. Nonetheless this route has become a popular solution among holders but lacks vital foresight.

WEAPONIZE PRIVACY COINS

Zero-knowledge proof systems based on legacy cryptography are an early and attractive target for attackers because they obfuscate activity. Compromised ZKP-based networks could allow an adversary to launder or liquidate stolen or illicitly created assets stealthily, avoiding immediate detection and undermining forensic traceability. There is also a concern the ZKP/proof system can have unknown quantum exploitable flaws which could lead to covert unknown minting. As ZKP constructs become more well understood it would be likely that more sophisticated exploits will arise. ZK BUG LIST - Zcash Exploit

BLOCK PRODUCTION (51%)

Block production would become centralized under the control of post-quantum computers capable of generating blocks significantly faster than classical miners. A quantum miner could dictate block contents, reorganize history, erase confirmed blocks, and capture all block rewards. However, these attacks are secondary compared to the impact of Shor’s algorithm, which alone could compromise the network’s integrity with far less effort than sustaining block-production dominance.

CROSS-CHAIN

Chains with cross-chain features, like swaps, open a vector where a single compromised chain can affect many others. This could effectively mint funds and generate endless swaps, even impacting chains that are

otherwise post-quantum secure. Post-quantum chains will need mechanisms to halt operations that depend on legacy networks to mitigate this risk.

FORKING & SDKS

Forking existing networks or SDKs whether Polkadot, Ethereum, Solana, or Bitcoin and simply swapping in post-quantum cryptography is a flawed approach. It fails to address the need for a fundamentally new architecture designed for a post-quantum world, leaving inherent limitations and vulnerabilities intact. Likewise, hastily adopting post-quantum ZKPs is no safe solution and introduces needless complexities; any meaningful design must be deliberate, rigorous, transparently verifiable, adaptable, efficient, and uncompromising to ensure true long-term security and adaptability.

EXISTING QUANTUM CLAIMED COINS

Some cryptocurrencies have introduced post-quantum “vaults” or isolated features. These isolated features become misrepresented as meaning the chain is quantum-safe even though the broader network still relies on legacy cryptography and wallets remain vulnerable. This has misinformed users to adopt systems such as Bitcoin Cash and others under the impression of post-quantum security, while in reality the chain, miners, consensus, and all wallets continue to depend on the same pre-quantum primitives.

The majority of Zcash wallets are transparent addresses, which expose public keys directly on-chain leaving them vulnerable to Quantum threats. Estimates suggest that roughly 70-77% of wallets use these transparent addresses and a vast majority of transactions flow through them, and they are not quantum-resistant. Zcash privacy pools are a different case and there is a notion that they may be more prepared but the math involved in the ZKP is still exploitable by a quantum threat. Exploits like counterfeiting in the pool are possible but there may be a degree of privacy for past transactions. When a ZKP minting exploit occurs Zcash intends to deactivate the privacy pool and invalidate all outgoing transactions further explaining legitimate funds could be lost forever.

For Bitcoin Cash, the situation is even more exposed: all wallets remain vulnerable to a quantum-capable adversary. A non-native, script-based "quantum vault" was recently proposed, but it does nothing to make the network itself quantum-resistant. The chain remains fully exploitable. Without a fully post-quantum chain, a quantum attacker could simply hijack miners and manipulate network consensus, rendering any vault protections meaningless. Even if the vault itself remained unaffected, the surrounding network would fail, likely collapsing the value of the assets it was designed to protect. The likely origin of the idea is from this older Bitcoin script article the script itself might also be a clone or borrowed from the original author.

NOT A SIMPLE ALGO SWAP

Post-quantum cryptography isn't just an algorithm upgrade, it shatters blockchain architecture. Larger signatures, massive keys, and heavy computation collide with monolithic designs that lack cryptographic agility. That additional post-quantum size reverberates through the stack: mempool pressure, P2P propagation latency and bandwidth, validator verification throughput, block packing, storage, and sync/bootstrapping times. Some networks, like Solana, aim to keep entire transactions under the MTU size, imposing transaction limits accordingly. This design becomes catastrophic in the post-quantum era, as modern NIST post-quantum public keys and signatures exceed the MTU, making such transactions infeasible.

These are not isolated crypto problems that can be patched with a library swap or arbitrary "quantum vaults". Q-Day represents a market-wide existential threat, and no current cryptocurrency is structurally prepared to withstand it. No current cryptocurrency is ready for what is coming.

NIST Challenges Associated with Adopting and Using Post-Quantum Cryptographic Algorithms

SOLUTIONS

STRATEGY OVERVIEW

Our strategy is designed to maximize legacy cryptography for high performance today, avoid post-quantum storage costs without relying on ZKPs, provide retroactive post-quantum protections for legacy wallets, enable automatic post-quantum transitions for all wallets, offer the most secure opt-in hybrid post-quantum protections for high-value long-term assets, and deliver a cutting-edge architecture fully prepared to perform in the post-quantum era.

CRYPTOGRAPHIC AGILITY

Introducing or enabling a new algorithm does not, by itself, provide meaningful cryptographic agility for a cryptocurrency. Real agility requires that every wallet can itself transition securely and consistently without fragmentation or risk, automatically. The existing wallet address must be preserved and protected. It must be an inherent property of wallets, not an optional feature applied only to new wallets forcing manual user based transition while the old wallet remains vulnerable.

To enable a secure transition for all pre-existing wallets to new post-quantum credentials ALL addresses MUST incorporate an inter-linked cryptographic mechanism which includes the following attributes:

- Address and future credentials must be inseparable
- Avoid upfront storage costs of post-quantum public key & signature
- Be stored as a compact cryptographic commitment
- Obscure future details
- Not incorporate the new address
- Rely on quantum-hard problems
- Permit systemic freezing of legacy keys
- Allow the legacy address to still function
- Retroactively secure assets

- Act as a standalone proof in the event only an address is on chain
- Use structured data as input
- Have unspent protections
- Doesn’t use the public key as an address

This capability must be inherent to address generation; without it, there is no immediate, systemic, cryptographically verifiable way to migrate to a new scheme. This process must ensure that all legacy wallet key pairs are disabled while **preserving full access** to the wallet’s **assets** and its **address**. This allows the entire network to be secured immediately, while individual wallets can transition to their post-quantum credentials at their own pace.

SCALE

To scale in the post-quantum world, a network must address storage concerns (storing, reading, and writing large cryptographic artifacts), maintain viable transaction throughput (TPS), optimize network protocols for post-quantum cryptography, and ensure cryptographic agility for future algorithm upgrades. Additionally, it must support massive parallel validation, efficient consensus under larger transaction sizes, and backward compatibility to transition legacy wallets without disrupting the network.

BUILT FOR QUANTUM

The post-quantum era can’t be secured by grafting new cryptography onto architectures built for the past Bitcoin, Solana, Ethereum, Polkadot. Legacy chains inherit structural assumptions that become hard limits in a quantum world. Forking or retrofitting these systems only delays failure and locks in long-term scalability ceilings Viat avoids those constraints entirely. It isn’t a fork or an adaptation it is engineered from the ground up for true quantum-era security and sustainable scalability.

SUMMARY

VIAT is designed to confront post-quantum threats, address post-quantum scalability challenges, enable efficient transitions, ensure cryptographic agility, maintain high performance today, avoid initial upfront storage costs of post-quantum public keys/signatures, and emerge as the most secure and capable post-quantum cryptocurrency after Q-Day.

OBJECTIVES

THE NEW STANDARD FOR THE QUANTUM ERA

Viat aims to be a **high-performance post-quantum cryptocurrency and ecosystem**, delivering post-quantum storage efficiency and scalability today while providing verifiable security against current and emerging quantum threats. The network is built for **cryptographic agility**, enabling wallets to seamlessly adopt more advanced or ideal post-quantum algorithms as standards evolve. Crucially, **all legacy wallets must remain fully functional and capable of transacting**, ensuring uninterrupted access to assets and preserving user trust. By combining performance, security, and forward-compatible design, Viat provides a robust, future-proof foundation for both everyday use and long-term value storage in the quantum era.

Post-quantum wallets need to be modular so they can adopt new cryptographic algorithms as stronger quantum computers are developed. Protecting against the first quantum computer that breaks legacy cryptography isn’t enough. To stay secure over time, already existing wallets must allow easy updates, support multiple layers of post-quantum security, and manage keys flexibly. This approach ensures users remain protected as quantum threats evolve.

VIAT

Viat /ˈviː.æt/ is a next-generation cryptocurrency built on a Hierarchical Proof-of-Work (H-PoW) architecture, designed for real-time validation, and seamless scalability. Each wallet maintains its own chain linked to global audit layers, creating a decentralized, self-organizing network. Viat operates efficiently with legacy cryptography while embedding a post-quantum trapdoor in every wallet address, allowing the entire network to upgrade automatically to future-proof security without migrations or loss of access. By treating every action as an independent verifiable block and supporting multiple chain types, Viat combines speed, adaptability, and long-term cryptographic resilience, ensuring all wallets and assets remain secure as quantum threats emerge.

Viat is not based on another blockchain network, SDK, and or a fork of another blockchain network. Viat is custom built from the ground up to be ready for the quantum era.

Viat's legacy wallets use standard cryptography for their primary key, but each is paired with a post-quantum trapdoor commitment that generates a secondary key. This makes every Viat wallet hybrid post-quantum and multi-key by default. The system can disable the primary key network-wide and shift seamlessly to the secondary "grace key" embedded in the commitment, or use that commitment to migrate to a newer post-quantum scheme when needed. This approach preserves assets and wallet addresses, provides retroactive protection against quantum attacks, and avoids the heavy storage overhead typically required when bootstrapping post-quantum adoption.

Viat intentionally avoids ZKP based designs, favoring faster execution and the well-understood security of NIST standardized primitives. ZKPs introduce complex, opaque components that, if compromised, can conceal vulnerabilities in both classical and quantum settings. Viat instead adopts a strict and transparent security model, ensuring that any failure is immediately visible and detectable across the entire network.

VIAT PROOF TYPES

Proofs in VIAT are dynamic and are open to change as the network grows and new features are implemented.

The following proofs are in VIAT:

- proof-of-work (ALL BLOCKS WITH minor PROOFS)
- Proof-of-funds (TX CREATION)
- proof-of-stake (STAKING BLOCKS REQUIRED TO MINE)
- proof-of-validation (ARBITERS)
- proof-of-history (AUDITORS)

THE CRYPTOGRAPHIC ALGORITHMS USED ARE:

- **LEGACY SIGNATURE SCHEME**: ed25519 (Primary LEGACY)
- **POST-QUANTUM SIGNATURE SCHEME**: Dilithium-44 (Trapdoor)
- **HASH**: SHAKE256 XOF (PRIMARY)
- **KEYED HASH:** KMAC256_XOF
- **STRICT 256bit HASH**: SHA3-256 (32-byte output)
- **STRICT 512bit HASH**: SHA3-512 (64-byte output)
- **POST-QUANTUM FAILSAFE**: SPHINCS+-128s (For Dilithium failure)

CRYPTO LINKS & SOURCES

- MODULE-LATTICE-BASED KEY-ENCAPSULATION MECHANISM STANDARD
- STATELESS HASH-BASED DIGITAL SIGNATURE STANDARD
- MODULE-LATTICE-BASED DIGITAL SIGNATURE STANDARD
- DIGITAL SIGNATURE STANDARD (DSS)
- RECOMMENDATIONS FOR KEY-ENCAPSULATION MECHANISMS
- TRANSITIONING THE USE OF CRYPTOGRAPHIC ALGORITHMS AND KEY LENGTHS
- ASD GUIDELINES FOR CRYPTOGRAPHY
- SHAKE256 INFO FIPS-202
- NIST POST-QUANTUM HOMEPAGE
- SHA-3 DERIVED FUNCTIONS
- TRANSITION TO POST-QUANTUM CRYPTOGRAPHY STANDARDS
- LIBSODIUM

- AEGIS-256
- HTTPS://PAULMILLR.COM/NOBLE/
- PQCLEAN
- AEGIS DOC
- AWS-POST-QUANTUM-CRYPTOGRAPHY-MIGRATION-PLAN
- CSA_CNSA_2.0
- WHAT-POST-QUANTUM-CRYPTOGRAPHY
- NIST THE ROAD AHEAD
- HTTPS://WWW.NCCOE.NIST.GOV/CRYPTO-AGILITY-CONSIDERATIONS-MIGRATING-POST-QUANTUM-CRYPTOGRAPHIC-ALGORITHMS
- NIST CHALLENGES ASSOCIATED WITH ADOPTING AND USING POST-QUANTUM CRYPTOGRAPHY

CRYPTOGRAPHY OVERVIEW

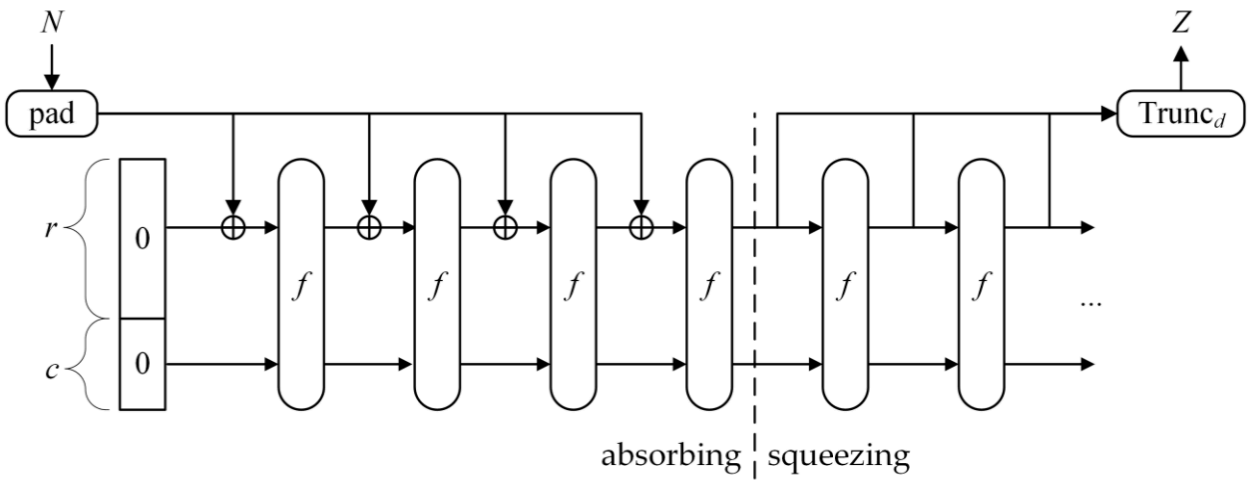
SHAKE256(M, d) = KECCAK[512] (M ‖ 1111, d)

"An XOF is a powerful new kind of cryptographic primitive that offers the flexibility to produce outputs with any desired length… However, XOFs have the potential for generating related outputs—a property that designers of security applications/protocols/systems may not expect of hash functions." - FIPS-202

	Security Strengths in Bits		
SHAKE256(*M*, *d*)	Collision	Preimage	2nd Preimage
LEGACY	min(d/2,256)	≥ min(d, 256)	min(d, 256)
QUANTUM	min(d/3,256)	≥min(d/2,256)	min(d/2,256)

Rate r = block size	Capacity c	Definition
1088	512	Keccak[512](M ‖ 1111, d)

SPONGE CONSTRUCTION: Z = SPONGE[Ƒ, PAD, R](N, D);



https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.202.pdf

Specification: Given an input bit string N and an output length d.

$$KECCAK[c](N,d) = SPONGE[KECCAK-p[1600,24], pad10*1, 1600-c](N,d)$$

ED25519

Ed25519's bit-security comes directly from the hardness of the Elliptic-Curve Discrete Logarithm Problem (ECDLP) on the edwards25519 curve. Given the Public Key A and the Base Point B, find the secret scalar a. This problem can be solved by Shor's algorithm and isn't quantum-hard resulting in a post-quantum bit security of "0".

DILITHIUM SIGNATURE SCHEME

$\underline{\mathsf{Gen}}$

$\mathbf{A} \leftarrow R_q^{k \times \ell}$

$(\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_\eta^\ell \times S_\eta^k$

$\mathbf{t} := \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$

$\mathbf{return}\ (pk = (\mathbf{A}, \mathbf{t}), sk = (\mathbf{A}, \mathbf{t}, \mathbf{s}_1, \mathbf{s}_2))$

$\underline{\mathsf{Sign}(sk, M)}$

$\mathbf{z} := \perp$

$\mathbf{while}\ \mathbf{z} = \perp\ \mathrm{do}$

$\quad \mathbf{y} \leftarrow S_{\gamma_1 - 1}^\ell$

$\quad \mathbf{w}_1 := \mathsf{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$

$\quad c \in B_\tau := \mathrm{H}(M \parallel \mathbf{w}_1)$

$\quad \mathbf{z} := \mathbf{y} + c\mathbf{s}_1$

$\quad \mathbf{if}\ \|\mathbf{z}\|_\infty \geq \gamma_1 - \beta\ \mathrm{or}\ \|\mathsf{LowBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2)\|_\infty \geq \gamma_2 - \beta,\ \mathrm{then}\ \mathbf{z} := \perp$

$\mathbf{return}\ \sigma = (\mathbf{z}, c)$

$\underline{\mathsf{Verify}(pk, M, \sigma = (\mathbf{z}, c))}$

$\mathbf{w}_1' := \mathsf{HighBits}(\mathbf{A}\mathbf{z} - c\mathbf{t}, 2\gamma_2)$

$\mathbf{if}\ \mathbf{return}\ [\![\|\mathbf{z}\|_\infty < \gamma_1 - \beta]\!]\ \mathbf{and}\ [\![c = \mathrm{H}(M \parallel \mathbf{w}_1')]\!]$

TEMPLATE FOR DILITHIUM SIGNATURE SCHEME

"Dilithium is a digital signature scheme that is strongly secure under chosen message attacks based on the hardness of lattice problems over module lattices. The security notion means that an adversary having access to a signing oracle cannot produce a signature of a message whose signature he hasn't yet seen, nor produce a different signature of a message that he already saw signed. Dilithium is one of the candidate algorithms submitted to the NIST post-quantum cryptography project." - DILITHIUM

ML-DSA (DILITHIUM) PARAMETER SETS

Parameters (see Sections 6.1 and 6.2 of this document)	Values assigned by each parameter set ML-DSA-44	ML-DSA-65	ML-DSA-87
q - modulus [see §6.1]	8380417	8380417	8380417
ζ - a 512th root of unity in $\mathbb{Z}_q$ [see §7.5]	1753	1753	1753
d - # of dropped bits from $\mathbf{t}$ [see §6.1]	13	13	13
τ - # of ± 1's in polynomial c [see §6.2]	39	49	60
λ - collision strength of $\tilde{c}$ [see §6.2]	128	192	256
γ_1 - coefficient range of $\mathbf{y}$ [see §6.2]	2^{17}	2^{19}	2^{19}
γ_2 - low-order rounding range [see §6.2]	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
(k, ℓ) - dimensions of $\mathbf{A}$ [see §6.1]	(4,4)	(6,5)	(8,7)
η - private key range [see §6.1]	2	4	2
$\beta = \tau \cdot \eta$ [see §6.2]	78	196	120
ω - max # of 1's in the hint $\mathbf{h}$ [see §6.2]	80	55	75
Challenge entropy $\log_2 \binom{256}{\tau} + \tau$ [see §6.2]	192	225	257
Repetitions (see explanation below)	4.25	5.1	3.85
Claimed security strength	Category 2	Category 3	Category 5

"We recommend using the Dilithium3 parameter set, which—according to a very conservative analysis—achieves more than 128 bits of security against all known classical and quantum attacks." - Dilithium Site

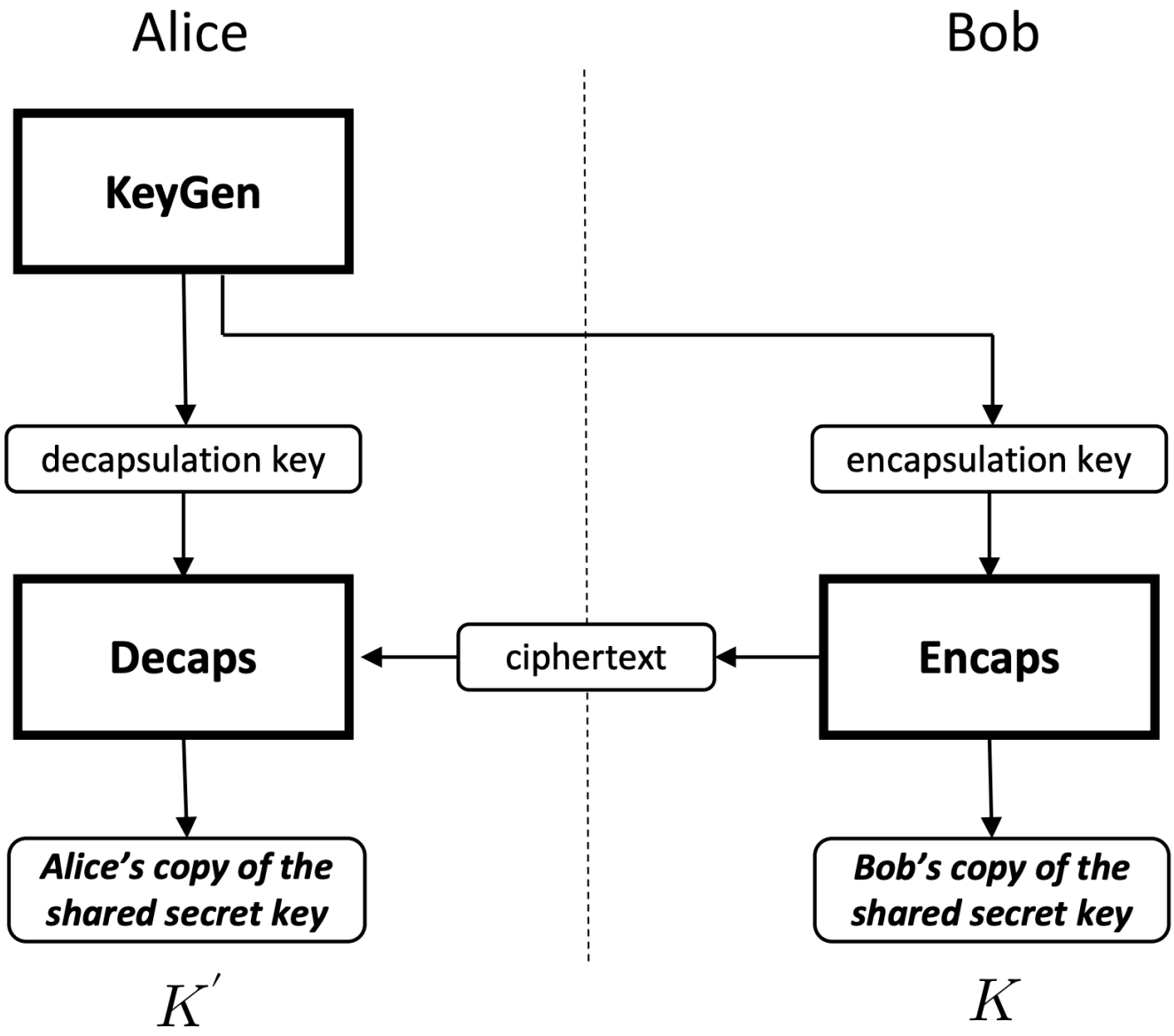
SPHINCS+ BACKUP

"SPHINCS+ is a stateless hash-based signature scheme, which was submitted to the NIST post-quantum crypto project. The design advances the SPHINCS signature scheme, which was presented at EUROCRYPT 2015. It incorporates multiple improvements, specifically aimed at reducing signature size. For a quick overview of the changes from SPHINCS to SPHINCS+ see the blog post by Andreas Hülsing. The submission proposes three different signature schemes:" - SPHINCS+

SLH-DSA-SHAKE-128s, a compact 128-bit variant of SPHINCS+, is implemented solely as a post-quantum failsafe for hybrid or fully post-quantum wallets in the event that **Dilithium** becomes compromised or deprecated. It serves as a worst-case contingency mechanism, as its large key and signature sizes make it impractical for standard wallet operations.

KYBER (ML-KEM)

Kyber is an IND-CCA2-secure key encapsulation mechanism (KEM), whose security is based on the hardness of solving the learning-with-errors (LWE) problem over module lattices. Kyber is one of the finalists in the NIST post-quantum cryptography project.



A simple view of key establishment using a KEM

"A KEM is a cryptographic scheme that, under certain conditions, can be used to establish a shared secret key between two communicating parties. This shared secret key can then be used for symmetric-key cryptography." - NIST

SUPPORTED POST-QUANTUM SIGNATURE SCHEMES

Additional signature schemes will be supported in the future; however, initial support focuses exclusively on standardized and NIST-approved algorithms. For a scheme to be considered viable, it must balance four key factors: minimal public key size, compact signatures, efficient signing, and fast verification, a combination not yet achieved by any single algorithm. The selected legacy wallet trapdoor signature scheme and primary post-quantum scheme, Dilithium-44 (ML-DSA-44), offers the optimal balance of these properties and meets Post-Quantum Security Category 2 standards.

POST-QUANTUM DIGITAL SIGNATURE ALGORITHMS				
NAME	Security	Public Key	Private Key	Signature
ML-DSA-44	128 Bits	1312 BYTES	2560 BYTES	2420 BYTES
ML-DSA-65	192 Bits	1952 BYTES	4032 BYTES	3309 BYTES
ML-DSA-87	256 Bits	2592 BYTES	4896 BYTES	4627 BYTES
SLH-DSA-SHAKE-128s (FAILSAFE)	128 Bits	32 BYTES	64 BYTES	7856 BYTES

WHY NOT ONE-TIME STATEFUL HASH BASED SIGNATURES

"Stateful HBS schemes are not suitable for general use because they require careful state management that is often difficult to assure, as summarized in Section 1.2 and described in detail in [8]." - NIST 800-208

Stateful one-time hash-based signatures carry an inherent operational fragility that NIST warns about. Their security depends on perfectly tracking which one-time keys have been used, and any mistake—whether from crashes, backups, concurrency, or system rollback—can lead to key reuse and immediate compromise. This strict state-management requirement makes them far riskier to deploy than modern stateless post-quantum signature schemes, and NIST therefore does not recommend them for most real-world systems.

There are other restrictions for such schemes such as max amount of possible signatures which effectively freezes a wallet. This ceiling on the number of valid signatures adds yet another operational constraint, making the scheme brittle in dynamic or long-lived deployments.

Given these constraints, any serious post-quantum cryptocurrency shouldn't rely on stateful one-time hash-based signature schemes regardless of perceived trade-offs. Arguments about signature size, aggregation, or public-key footprint don't overcome the fundamental trade-off: strict state tracking, limited signature capacity, and the risk of catastrophic key reuse. NIST-selected post-quantum algorithms provide balanced, modern alternatives without these structural liabilities. With a well-architected system, their larger sizes can be absorbed, pruned, and or isolated as one-time costs, offering long-term security without operational fragility.

FUTURE POST-QUANTUM ALGORITHMS

Viat is designed to quickly adopt any post-quantum signature scheme or key exchange with smaller public key and signature sizes by using its existing address based post-quantum commitment scheme (Trapdoor). If the commitment is satisfied the trapdoor does allow a 3rd option which can be added to Viat later such as SQIsign, ensuring wallet based cryptographic agility.

Below are algorithms or proposals we are reviewing:

- ISOGENY BASED CRYPTO
- https://sqisign.org (SMALL SIGNATURES & KEYS)

Isogeny-based post-quantum algorithms currently offer some of the most compact key and signature sizes. However, several early isogeny schemes, such as SIDH, have already been broken, and others may follow as cryptanalysis advances. Among signature schemes, SQIsign v2 presently shows the most promise for compactness. Historically, cryptography for efficient network communications has trended toward reduced key and signature sizes as new algorithms mature, and this pattern is likely to continue in the post-quantum era.

SQIsign v2.0.1

LEVEL	PUBLIC KEY	SIGNATURE
NIST-I	65	148
NIST-III	97	224
NIST-V	129	292

Not viable for use yet needs maturity, improvements, and cryptanalysis.

WALLETS & ADDRESSES

INTRO

Each address contains a forward-secure “trapdoor” (a post-quantum cryptographic-commitment scheme), allowing every wallet to transition seamlessly to a post-quantum key pair without manual wallet migration, loss of assets, or loss of address. This upgrade path is embedded from the beginning, ensuring that when a hint of Q-Day arrives, the network can securely and verifiably transition all legacy wallets in place. The trapdoor provides retroactive post-quantum protections to legacy wallet assets and users retain access to the legacy wallet’s address without upfront post-quantum storage costs.

The trapdoor functions as a built-in, time-delayed multisig path for every wallet. When a wallet is created, it commits to a post-quantum trapdoor value that is hidden but verifiable. This commitment represents an alternate keypair that is not active yet, but can be activated later by revealing the trapdoor.

```
WALLET = { KIND , VERSION, CIPHER, PUBLIC_KEY };
ADDRESS = XOF( { …WALLET , XOF(TRAPDOOR) } , OUTPUT_SIZE );
```

The address generation process enforces strong domain separation, modular upgrades, includes a quantum-ready failsafe, enables shortcut address generation, includes inherit cryptographic agility, and permits address expansion.

Each Viat address is constructed from two components:

1. **Primary Structure:** Derived from the wallet’s main key pair & it’s meta details.
2. **Trapdoor:** A compact commitment derived from the secondary security key pair & it’s meta details

POST QUANTUM TRAPDOOR (ENTROPY (ORDER))

COMMITMENT SCHEME

"Commitment schemes arise out of the need for parties to commit to a choice or value and later communicate that value to the other parties involved in such a way that is fair to all the parties."- Introduction to Cryptography

PROOF OF CRYPTOGRAPHIC CONTINUITY

The trapdoor hash is a post-quantum commitment scheme which obscures the post-quantum public key. When the user eventually reveals the trapdoor, the commitment is satisfied and the trapdoor key pair becomes the new primary key pair for the same wallet address. This creates a seamless upgrade path: the wallet transitions from its current keys to new PQ-secure keys without losing its address or breaking historical links.

As a result, every Viat wallet is effectively a delayed multisig wallet by default one primary key controlling the present, and one committed key sealed behind a trapdoor for future activation. This built-in cryptographic agility allows any wallet to upgrade its security model or cryptographic algorithm as standards evolve, without requiring migrations, forks, or new addresses. Smart contracts can continue to send to the original wallet address securely and uninterrupted.

```
TRAPDOOR_STRUCTURE = { KIND , VERSION, CIPHER, PUBLIC_KEY };
TRAPDOOR = XOF( TRAPDOOR_STRUCTURE , OUTPUT_SIZE );
```

TRAPDOOR ACTIVATION PROCESS

When the trapdoor is activated, the original primary key pair is permanently disabled and the trapdoor key material becomes the basis for transitioning the wallet to a new state or new owner. Demonstrating knowledge of the trapdoor and its associated private keys proves legitimate control and authorizes the transfer.

The mechanism also applies to unspent addresses that only have a receiving address recorded on-chain. For these solo addresses, the resulting proof is larger because it must include the full wallet construction data rather than referencing an existing on-chain wallet block.

SCHEME VERSION: 1.0.0

```
STRUCT = DOMAIN( OLD_ADDRESS | NEW_ADDRESS | TRAPDOOR );
OLD_SIG = SIGN( OLD_KEY, STRUCT );
NEW_SIG = SIGN( NEW_KEY, STRUCT );
SIGS = { OLD_SIG, NEW_SIG };
WALLETS = { OLD_WALLET, NEW_WALLET };
PROOF = PROVE( WALLETS , SIGS, TRAPDOOR_ORIGIN );
FINAL = SIGN(NEW_SIG, BLOCK( BLOCK_META… | PROOF ) );
```

A trapdoor activation request may specify a different cryptographic algorithm, allowing the wallet to transition directly into a newer scheme rather than the one originally embedded in the trapdoor. This built-in agility is essential for adopting future post-quantum algorithms that may be more compact or more advanced than the trapdoor's initial choice.

```
STRUCT = DOMAIN( OLD-ADDRESS | NEW-ADDRESS | TRAPDOOR );
OLD_SIG = SIGN( OLD_KEY, STRUCT );
TRAPDOOR_SIG = SIGN( TRAPDOOR_KEY, STRUCT );
NEW_SIG = SIGN( NEW_KEY, STRUCT );
SIGS = { OLD_SIG, NEW_SIG, TRAPDOOR_SIG };
WALLETS = { OLD_WALLET, NEW_WALLET };
PROOF = PROVE( WALLETS, SIGS , TRAPDOOR_ORIGIN );
FINAL = SIGN(NEW_SIG, BLOCK( BLOCK_META… | PROOF ) );
```

PRESERVED LEGACY ADDRESS ACCESS

The wallet address with an activated trapdoor remains intact and reserved for receiving. Prior smart contracts or app integrations continue to function as expected. Any assets sent to it can still be directly transferred to the new wallet address or can act as a proxy for the new address.

UPGRADING TRAPDOOR SECURITY MARGIN

If quantum computers increase faster than expected the most immediate strategy for trapdoors is straight forward. The security of the trapdoor can be increased in two ways: by using a 64 byte SHAKE256 output to reach SHAKE256's claimed security margin, and by increasing the Dilithium NIST level to enhance lattice hardness. The combination of a robust hash function and a higher-security lattice scheme ensures strong post-quantum security for long-term protections.

LEGACY WALLET VAULT MODE

Early activation of the trapdoor can at a greater upfront transaction cost turn a legacy address into a high-security post-quantum vault. Part of the request includes initial config to specify limitations, script logic, rules, failsafe trapdoor, and new owners.

MODULAR TRAPDOORS

A modular trapdoor is the same trapdoor hash but stored inside of a CBOR structure alongside meta details in plain-text. These meta details such as CIPHER_ID must match the details used to generate the trapdoor hash. This permits a consistent hash size to be used with various algorithms. For example if there is a collision for the trapdoor hash but the CIPHER_ID is different then both commitments can still be satisfied and securely verified.

ADDRESS FORMATS & MODULATION

Viat wallet addresses support **dynamic sizing**, allowing extremely short addresses for quick sharing. This feature is intended primarily as a temporary mechanism, as collision risk increases sharply as the hash output size decreases. Implementors supporting temporary shortened addresses must take collision risk into account.

If the same key pair requires a larger format, the network can upgrade all addresses due to the underlying address derivation process and the use of an XOF hashing function. Shortened addresses are generated by setting a reduced SHAKE256 size output or truncation of a larger

SHAKE256 hash, which the network can use for address lookup. Only hashes which were constructed with an XOF can do this properly otherwise it's basic truncation of a specific hash.

Viat also supports **filesystem-style address formats** (e.g., "aa/bb/cc/1…19"), providing a structured path that enables efficient, file path-friendly block lookups. The hex-based address filepath structure enables the filesystem to organize blockchain data hierarchically, allowing efficient direct lookup of any block by its structured path. The file path structure is designed for use on modern variations of B-Tree filesystems on modern high-bandwidth solid state storage such as NVMe. This format might not be applicable to all nodes as hardware and configuration will dictate storage strategy.

ADDRESS TYPES

There are different wallet types all with varying degrees of quantum security, threat models, and strategies. All wallets contain proactive measures to prepare for the inevitable rise of the post-quantum era.

Address formats will need to evolve continuously, not just with the first wave of quantum computers. By embedding trapdoors in all addresses, Viat ensures that wallets and their schemes can adapt automatically, allowing the chain to endure and evolve securely alongside humanity as computational threats advance.

LEGACY ADDRESS (LA)

ADDRESS SIZE: 24 BYTES
SCHEME: ed25519 ⟶ Dilithium Trapdoor

A legacy address uses an Ed25519 key pair as its primary signing keys, paired with a built-in Dilithium trapdoor. The trapdoor can be activated in the event of Q-Day or when placing funds into a long-term quantum-secure vault state. As quantum capability increases, the network will introduce additional features for existing trapdoors.

Legacy wallets use a hybrid proactive–reactive post-quantum security model. They remain lightweight and inexpensive to operate under normal conditions, but include a built-in mechanism to transition into a post-quantum wallet or a post-quantum vault state when required.

HYBRID-POST-QUANTUM ADDRESS (HQA)

ADDRESS SIZE: 32 BYTES
SCHEME: ed25519 & Dilithium ⟶ Dilithium || SPHINCS+ Trapdoor

A hybrid wallet uses two primary key pairs: an Ed25519 key and a post-quantum trapdoor key (Dilithium or SPHINCS+). This wallet type is initially reserved for specific roles such as auditors and will be handled on a request basis after which rollout will begin for network wide usage.

To avoid cryptographic coupling, each algorithm independently signs the same message, producing two distinct signatures rather than one wrapped in the other. This preserves clear verification semantics and future flexibility: the Ed25519 signature may be deprecated or ignored when appropriate while the post-quantum signature continues to provide verifiable security.

POST-QUANTUM ADDRESS (QA)

ADDRESS SIZE: 40 - 64 BYTES+
SCHEME: Dilithium ⟶ (Dilithium OR SPHINCS+) Trapdoor

A post-quantum wallet uses Dilithium as its default signature scheme and embeds a SPHINCS+ trapdoor for failsafe security. These post-quantum wallets are limited in availability and reserved for system-level wallets, long-term quantum vaults, government strategic reserves, cryptocurrency exchanges, custodial stores, and other high-value assets requiring verifiable, standardized post-quantum security for long-term protection.

ADDRESS SECURITY

The security of wallet addresses is computationally binding but the full potential security level depends on the algorithms used, hash sizes, the attackers capabilities, known details, and the attack vector. An unbounded adversary *could* potentially break the binding (e.g., find a collision), but doing so would be computationally infeasible in the real world (requiring an exponential amount of time or resources). Address size can be dynamically expanded ensuring attacks are increasingly difficult for specific scenarios.

BREAKDOWN OF LEGACY ADDRESS SECURITY

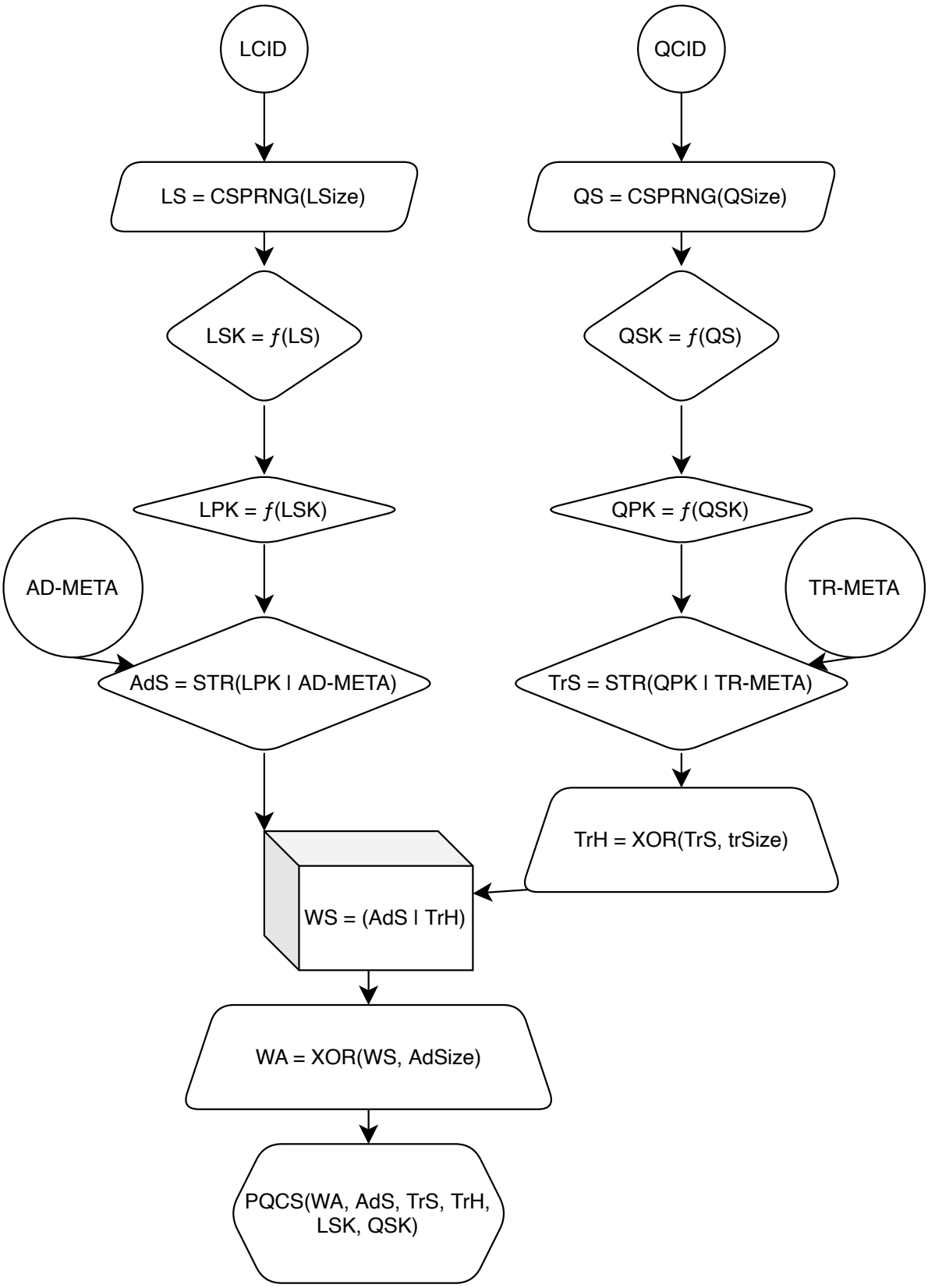
Without a wallet block it requires an attacker to find a matching 32 byte value (which represents the ed25519 public key then the private key can be computed with Shor) that when hashed with your chosen trapdoor hash results in the same address. If the wallet block is on chain and

the ed25519 key pair is broken then this requires generating a Dilithium key pair with a public key that results in the same 32 byte trapdoor hash output. Both attacks are computationally binding.

LEGACY WALLET PRIMITIVE SECURITY BREAK DOWN			
STEP	ALGO	Legacy Security	Quantum Security
SHAKE256/192	GROVER COLLISION	96 Bits (collision)	~64 Bits
Ed25519	SHOR (ECDLP)	128 Bits	0Bits
SHAKE256/256	GROVER COLLISION	128 Bits (collision)	~85 Bits
ML-DSA-65	Module-LWE, Module-SIS	128 Bits	128 Bits

VIAT WALLET ADDRESS GENERATION & POST-QUANTUM-COMMITMENT-SCHEME

QSK & QPK can be seen as a single operation depending on the primitive others can derive the PK from the PS.



SEED BASED HIERARCHAL DETERMINISTIC WALLETS

"For both FIPS 203 and FIPS 204, a KeyGen seed is considered an acceptable alternative format for a key-pair, or for the private (i.e., decapsulation or signing) key. In particular, generating the seed in one cryptographic module and then importing or exporting it into another cryptographic module is allowed. The internal key generation functions ML-KEM.KeyGen_Internal(d, z) and ML-DSA.KeyGen_internal(ξ) can be accessed for this purpose." - https://csrc.nist.gov/projects/post-quantum-cryptography/faqs

Viat replaces key-based derivation with a seed-based model: the Hierarchical Deterministic Seed Tree (HDST). Rather than deriving child keys from a parent key, HDST deterministically generates isolated child seeds, each of which is then fed directly into the internal key generation function of its target post-quantum as permitted by NIST FIPS 203 and FIPS 204. This means every wallet, signing key pair, key-exchange key pair, and symmetric key in the hierarchy originates from the same root of trust yet remains cryptographically independent, no child seed reveals any information about its siblings or master.

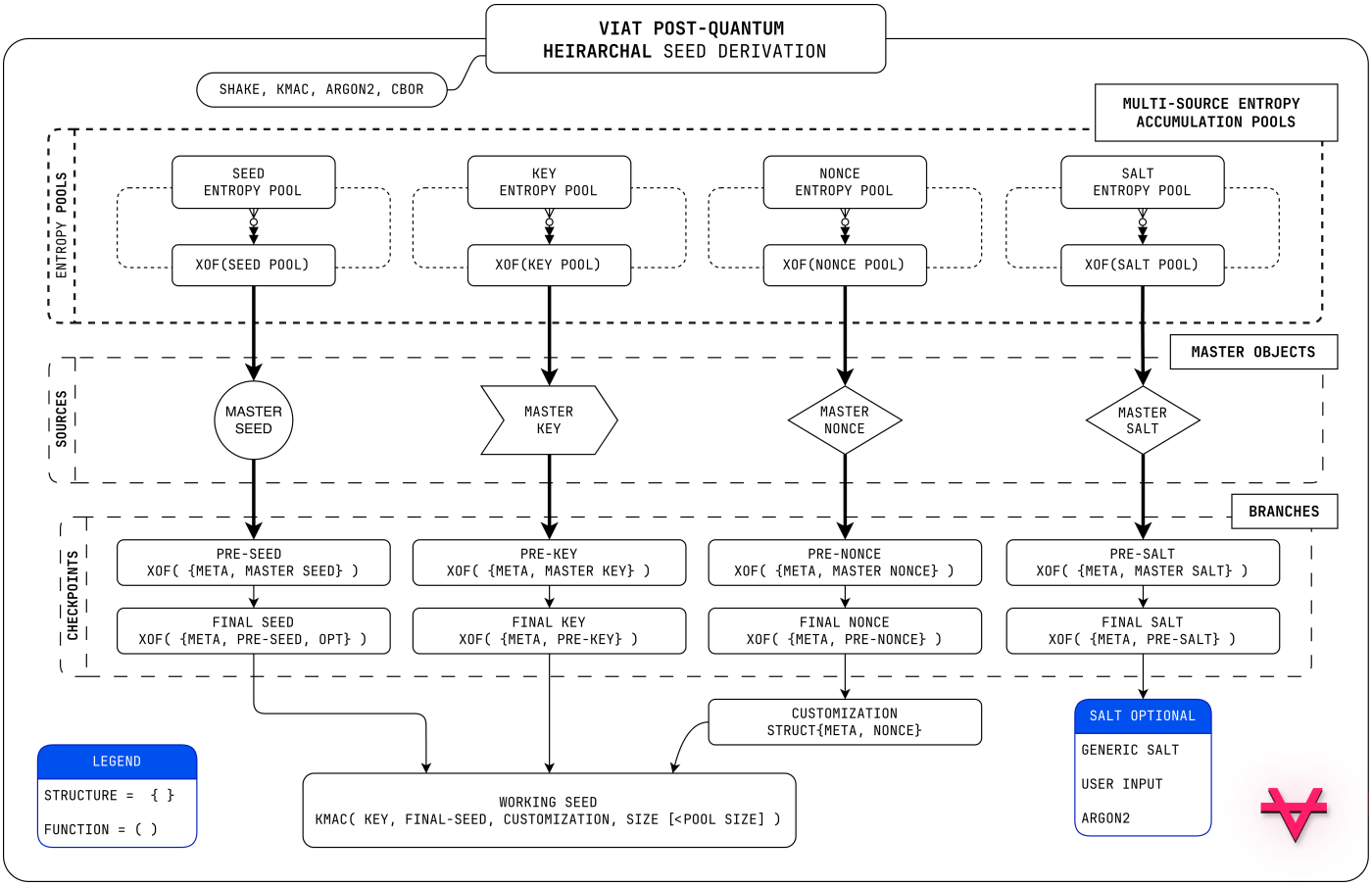
HDST consists of four multi-source independent master entropy pools: a Master Seed, Master Key, Master Nonce, and Master Salt. Entropy pools used in child derivation are statistically independent from each other, eliminating single-point-of-compromise risk: exposure of any one pool does not compromise the others.

Child seed derivation follows a multi-stage pipeline with strict domain separation. Each derivation incorporates contextual metadata such as the target cryptographic scheme, intended purpose, position in the hierarchy, and network parameters, ensuring derivations differing by even a single parameter produce completely unrelated output. The pipeline employs NIST-standard cryptographic primitives including SHAKE-256 and KMAC-256-XOF throughout, with deterministic binary serialization guaranteeing identical results across any platform.

This architecture supports both vertical derivation (parent to child chains) and horizontal derivation (sibling seeds at the same level), enabling a tree of wallets, identities, sessions, and credentials from a

single set of master secrets. Each derivation is scheme-aware, producing output sized to its target algorithm's requirements.

Additionally, every derivation can produce a companion trapdoor seed, allowing each wallet to carry a built-in post-quantum commitment from the moment of creation. HDST is not limited to cryptocurrency wallets — it generalizes to universal identity, authentication, encryption keys, account logins, web apps, operating system user accounts, and any context requiring deterministic, hierarchical key material.



https://github.com/universalweb/Network/blob/master/docs/diagrams/viat/HDSeed.png

Viat's HDST leverages SHAKE256, CBOR, AEAD constructs, memory-hard hashing functions for mixing in optional user based input, and KMAC256_XOF to produce a chain of fully isolated wallet seeds.

WALLET CHAIN

INTRO

Each wallet in Viat maintains its own dedicated blockchain composed of specialized chain types. Linear chains for sequential operations and DAGs for parallelized receipts, each optimized for its specific function. Every wallet chain begins with a cryptographic wallet anchor, a deterministically generated block that serves as the immutable root of that wallet's history.

ARCHITECTURE

Unlike monolithic blockchains that rely on a single global transaction ledger, Viat decentralizes transaction storage into independent wallet chains. These wallet chains and their blocks are later referenced and ordered within the global state through **audit blocks**, enabling scalable, asynchronous validation without sacrificing consistency or immutability.

LINEAR OUTGOING

Outgoing transactions are recorded on a linear chain composed of blocks created and signed by the wallet owner. This structure enforces strict transaction ordering, prevents double spends, and simplifies collision handling. Because forks are confined to individual wallet chains, any attempt to manipulate transaction history or induce collisions is isolated and easily identifiable allowing deterministic resolution without impacting the broader network.

To travel down the chain for a transaction we follow this:
[wallet anchor → wallet block → Transaction Chain → TX0]

DAG INCOMING

Incoming transactions are recorded within a **DAG (Directed Acyclic Graph)** structure governed by distinct write rules. Each receipt block in the DAG is created by the sender of the transaction and must reference either an existing receipt or the wallet's anchor block. This architecture enables concurrent validation and parallel appending of multiple trans-

actions, dramatically increasing throughput and allowing the network to scale without contention between independent wallet chains.

To travel down the chain for a receipt we follow this:
[wallet anchor → receipt anchor → Receipt DAG → RC0]

COLLISIONS
Any collisions or forks occurring would stem from individual wallet chains which means the wallets involved were intentionally causing the collision within their own chains. Therefore, the collision is intentional, isolated, and quickly resolved deterministically.

BASIC BLOCKS

All blocks in VIAT are CBOR encoded, self-contained atomic structures. The system defines two primary block types: deterministic blocks and signed blocks. Deterministic blocks can be generated dynamically from predefined data, while signed blocks require a valid cryptographic signature from their creator.

A VIAT block is constructed into a modular canonical CBOR object with three main sections:

- The primary section contains the **DATA** part of the block where all block details are located.
- The second section is the block's **HASH**
- If present the last section is the block's **SIGNATURE**
  - Signatures are not included in the hash which enables signature pruning - Required for large PQ Signatures

Within the DATA section there are two main sections:

- The first section is the **META** which includes basic information such as timestamp, block type, version, format, and so on.
- The second section is the **CORE** which contains transaction specific details.

LINEAR BLOCK SIGNATURE AGGREGATION

Due to the linear nature of the transaction chain many blocks can be attested to with only one signature and or proof. This is a required aspect for an efficient post-quantum blockchain that directly addresses processing, validation, and most of all storage costs. Lighter clients can choose to prune older signatures to free up storage for new data.

ANCHORS

Anchor blocks are deterministically generated structures that serve as foundational connection points for other blocks or as links between distant yet related data. For instance, a **wallet anchor** enables a wallet address to receive funds even before the wallet owner has created an active wallet block, allowing seamless initialization and interoperability across the network.

WALLET BLOCK

A wallet block contains the wallet's essential cryptographic components, including public keys, the wallet address, signature, trapdoor, and related metadata. Wallet blocks are a type of signature block and must be signed by their owner before they can be processed and stored on the network. Wallet blocks also help to reserve an address which means any potential collisions wouldn't result in exploitation.

Wallet Blocks serve as the origin point for all subsequent signature blocks within a wallet chain, every signed block ultimately traces its lineage back to a wallet block. To reference a transaction's public key, the network uses the wallet address to locate its associated wallet block, which holds the public key. This approach ensures that large post-quantum public keys are stored only once, minimizing redundancy and optimizing on-chain storage efficiency.

Viat address generation is co-designed with Wallet Blocks. Address generation and Wallet Blocks function as a unified system, where the ad-

dress encodes only the minimal cryptographic commitments, while the Wallet Block stores the associated metadata and key material extensions on-chain. This co-designed scheme yields compact, quantum-ready addresses without requiring bulky proofs or large on-chain state, enabling the Wallet Block to provide the extended wallet details only when needed.

A wallet block is a prerequisite for all wallet activity. No transaction or signed operation can occur without one. Any action that introduces a signature block must first include a corresponding wallet block creation request, ensuring that the first signature block in a chain is securely linked to its wallet block.

TRANSACTION BLOCK

Stored in a linear chain a transaction block contains all immutable details of the senders side of the transaction and proves that the transaction is authentic, possible, and valid. Each transaction block has a numerically incrementing ID which is strictly enforced. Transactions referencing invalid prior TX blocks are automatically invalid. Order is strictly enforced per wallet. The block data proves a transaction is possible, has the required funds, and doesn’t double-spend. The transaction block is used to generate the associated receipt block’s base deterministic details.

Viat uses a hybrid UTXO model with a sweep-and-condense receipt strategy. Each transaction must include up to X prior receipts as inputs even when this exceeds the amount being spent. This ensures all older receipts are collapsed into one fresh output. This naturally cleans up historical receipts and makes transaction flow self-optimizing for both clients and validators. By pushing this behavior to the client side, the network encourages users to retire old receipts, which reduces lookup overhead and accelerates future transaction processing for that particular wallet. Miners can then archive or discard many older receipts which frees up memory for a newly created consolidated receipt. Once older receipts are consumed they can be moved to cold storage. Lightweight miners can choose to remove them entirely and only keep a per-wallet chain

bloom filter, append-only-log, and or compact summary of unspent receipts. Isolating UTXO data into per-wallet chains drastically reduces the search space and simplifies the logic required to process each wallet's UTXO set, rather than operating across a global pool.

Transaction blocks can support general scripting operations which can enforce either a signature, script logic, and or both in order for a transaction to be fully processed.

RECEIPT BLOCK

Stored in a **DAG chain**, receipt blocks serve as the account-specific equivalent of UTXOs and provide confirmation of a transaction. Each receipt block contains key details duplicated from the corresponding transaction block, creating built-in redundancy. Receipt blocks link the receiving wallet to the sender's wallet, reference prior receipt blocks, reference the associated transaction block, light client account sync, and enable **per-wallet real-time transaction processing**.

SYSTEM BLOCKS (LEVEL 0 & LEVEL 1)

The first two levels of VIAT's superstructure, specifically LEVEL 0 and LEVEL 1, incorporate system blocks. On LEVEL 0 there is only the genesis block from which all blocks link back to. LEVEL 1 is reserved for core blocks which start all major branches in the superstructure. This forms the initial geometry of the superstructure and it's main branches that act as deterministic origin anchoring points. All none system blocks link back to LEVEL 1 Core Blocks.

Core Blocks represent the primary branches of the blockchain superstructure, each defining its own ruleset for appending new blocks. The **Wallet Core Block** serves as the anchor point for all wallet chains through their respective wallet anchor blocks. Consequently, all transaction blocks fall within the scope of the Wallet Core Block, also referred to as the *wallet branch*. Another example is the **Audit Core Block**, a linear,

strictly enforced chain of uniquely identified blocks that forms the canonical backbone of the network.

"If d > r + c/2, then SHAKE128 and SHAKE256 provide more than 128 and 256 bits of preimage resistance, respectively; moreover, if d > 1600, a preimage probably does not exist." - FIP 202

In Viat, system block hashes use a deliberately large output from SHAKE256 exceeding 200 bytes, larger than any other block in the network. This is done for the following reasons:

1. Strong Cryptographic Anchor & blockchain SEED
    - System Blocks serve as the foundation of the entire blockchain/superstructure. The large hash output strengthens the cryptographic uniqueness of System Blocks, ensuring that no alternate configuration can replicate its root identifier through feasible computation.
2. Deterministic and Verifiable
    - Every node can **recompute the system block hashes from the hardcoded data**, ensuring that all participants start from the same secure anchor.

BLOCK SPAM PREVENTION

Wallet blocks must be submitted together with at least one corresponding signature block within the wallet's local chain. If a wallet block is broadcast without a linked signature block such as a transaction block, nodes may safely discard it. This requirement provides inherent spam resistance and flow control by enforcing a minimal initial computation cost on the block creator. By introducing this upfront cost, the system mitigates cheap storage attacks, which is a critical defense strategy in post-quantum decentralized environments.

MINING

Mining splits into two coordinated stages. Arbiters handle the validation phase, acting as a high-throughput, decentralized filter that veri-

fies transactions and provides probabilistic finality. Auditors handle the canonical phase, ordering validated transactions into the definitive historical record. This design allows the network to scale like a high-performance mempool while still producing a canonical, by-hash-referenced block history. Think of arbiters as the House of Representatives while the Auditors are the Senate. They both have their own responsibilities and limitations.

Two types of miners:

- **Arbiters:** Realtime transaction verification, submitting arbiter blocks (proof of verification), and enforcing STATE_ID.
- **Auditors:** Collect valid blocks, announce STATE_ID changes, create audit blocks (historical maps), and maintain canonical linear history.

TIERED MINING DESIGN BENEFITS:

- Decentralized block creation and validation
- Highest decentralized TPS
- Separation of powers
- Arbiters Mitigate SPAM & bad requests
- Flow control
- Reduction of attack vectors on core miners
- Can support true fast lanes

MINER STAKING

Staking on VIAT is not used to determine block production like traditional proof-of-stake. Instead, it functions as a security deposit, creating a financial incentive for miners to act in good faith. Miners must still perform useful work to participate, and misbehavior risks slashing their staked funds. Arbiters stake but alone do not construct canonical blocks or potentially any blocks. Auditors who have staked do obtain the rights to create a block but it doesn’t determine if the auditor’s block will be selected.

Each stake is a time-bounded session represented by a staking block that automatically expires at the end of its cycle. Miners can renew their participation by issuing a new transaction block that adds fresh staking funds before the previous session closes. To withdraw or end a stake early, a miner creates a transaction that references the original staking action, producing a receipt on their own receipt DAG returning the remaining funds.

The initial required staking amount is hardcoded during the bootstrapping phase of VIAT. As the network grows the staking amount can fluctuate and if activity is too low then alternative rules are observed and a new audit block can be used to continue operations.

This event-driven model provides various benefits:, keeps staking accountable, keeps miner pool fresh, and continually recycles VIAT back into the network.

- Prevents indefinite staking claims
- Enforces that every stake ends with a concrete receipt
- Prevents spamming
- Reduces collusion potential
- Keeps arbiters and auditors fresh, active, & alive
- Avoids automatic system actions

SLASHING

- Occurs via provable malicious proofs (double-spend or manipulation). Deferred reporting ensures fairness.
- Slashing helps to recycle Viat back into the mineable total extending the emissions timeline. Reporting truthfully benefits all auditors and arbiters.
- Slashing can be modified to distribute slashed funds to existing miners which encourages the network to report bad actors.

MINING DOMAIN NAME

Arbiters and auditors must register a Universal Web domain before they can begin mining. The domain certificate integrates directly with the Domain Information System, giving miners a fast, authenticated network identity and allowing connections to establish with far fewer packets than traditional protocols (HTTP, TCP, QUiC). Because UDSP and DIS fuse routing, identity, and encryption into a single flow (authentication-via-encryption), miners benefit from consistently faster request–response cycles an essential ingredient for real-time decentralized validation.

Because every miner is tied to a verifiable domain identity, misbehavior becomes easy to trace, and malicious or unreliable nodes can be isolated quickly, giving the network a built-in layer of accountability.

The recurring renewal model for UW domains provides a predictable revenue stream that helps fund core network operations much like domains on the World Wide Web.

ARBITERS

An **Arbiter** is a special type of miner responsible for **real-time transaction verification and STATE enforcement**. Arbiters create proofs which demonstrate they have validated a transaction and sign off on the validity of its contents and their proof. You must stake VIAT to be considered as an Arbiter any arbiter blocks submitted to the network which hasn’t staked a block is automatically denied.

Think of arbiters as a decentralized high-performance mempool dedicated to transaction validation.

Arbiters serve as the first decentralized filtering layer, distinguishing valid transactions from invalid ones before they enter the broader network and eventually auditors. By doing so, they reduce unnecessary propagation of bad data and help maintain network integrity. Beyond

strict validation, Arbiters also exercise discretionary control: they can choose to deny processing requests for any reason, whether due to resource limitations, suspected malicious activity, or simply prioritizing higher-value transactions. This flexibility allows Arbiters to dynamically manage high network load, optimize their own computational and bandwidth resources, and provide a resilient buffer against spam or denial-of-service attempts.

When auditors issue a STATE_ID change, arbiters enforce it network-wide by rejecting any request that doesn't reference the correct value. This ensures new transactions remain consistent and fresh. If a client submits a transaction with an outdated STATE_ID, the arbiter immediately returns an UPDATE_STATE_ID in the response, allowing the client to correct and resubmit without disruption.

ARBITER BLOCKS

An arbiter block acts as a vote on an individual transaction and includes a small proof showing they verified the transaction and attest to the validity of the transaction. Arbiter blocks have two ways to be verified: if it's directly from the source Authentication-via-encryption can be used if stateless authentication is required a signature is used. Arbiter blocks are not commonly stored but in the event of slashing they must be stored with a signature. Only one arbiter block can be produced for a given transaction multiple attempts can be seen as fraudulent or spamming and slashing may result. Arbiter blocks if stored are part of an offshoot DAG chain attached to the transaction block.

ARBITER REWARDS

Arbiter rewards are distributed through a deterministic hash-distance mechanism per-transaction. Then combined with proportional compensation based on the total validation work performed during a cycle.

Each validated transaction enters a hash-distance lottery. The arbiter's block hash is compared against the transaction hash, and rewards are assigned to the arbiters whose hashes are closest in value. Rewards decline for each subsequent position in the distance ranking until the al-

location is exhausted. Any residual amount that cannot be evenly distributed is recycled back into the system's mineable amount. This approach gives smaller or lower-capacity arbiters a fair opportunity to earn rewards even when they cannot process large volumes of transactions.

A fixed minority percentage of each mined block is also reserved for proportional distribution. This portion is allocated according to the total number of transactions validated by each arbiter during the cycle, with rewards assigned in descending order. This ensures that arbiters who perform the most work receive consistent and meaningful compensation, even in periods of high arbiter participation where per-transaction lottery outcomes may vary.

If an arbiter validates a transaction that does not reach the required quorum of arbiter confirmations, that transaction will not be eligible for inclusion in an audit block. In such cases, the arbiter does not receive the associated transaction fee. The required quorum can adjust dynamically based on the number of active arbiters known prior to each cycle. If storage constraints become less significant in future implementations, this model could be expanded to allow failed transactions to be retained which do have sufficient TX funds. In that scenario, arbiters could still receive the associated transaction fee when applicable. However, many failed or fraudulent transactions would not provide a fee at all, which limits the practicality of compensating arbiters for such cases.

AUDITORS

Auditors coordinate global state progression by announcing STATE_ID transitions and consolidating all arbiter-validated transactions. Each audit cycle produces an audit block that references these validated records, establishing a canonical ordered history for the network. This process ensures that local activity across wallet chains is coherently anchored into a unified global state.

AUDITOR REWARDS

Auditors receive a substantial portion of the Viat minted from each audit block. Rewards are allocated through a deterministic hash-distance lottery. The sync-trie root hash serves as the reference value. For each auditor, the hash of their address combined with the sync-trie root is compared against the root hash itself, and rewards are distributed in descending order of proximity until the allocated pool is exhausted or a small remainder is returned for recycling. This approach keeps distribution deterministic, computationally efficient, difficult to manipulate, and broadly accessible, emphasizing contribution and correctness rather than competition or timing advantages.

LEADER SELECTION POOL

Because audit block ids can be precomputed for future state transitions, auditors can identify the leaders for upcoming cycles. The auditors whose combined hashes are closest to the sync-trie root hash effectively act as designated leaders for the next audit block. If the closest auditor fails to submit a valid audit block, the primary award cascades to the next closest, and so on. Given enough historical data post-bootstrap phase auditor performance can be incorporated into more effective leader pool selection ordering.

Only auditors who submit a valid proof or provide a vote are eligible for rewards. It is possible for some auditors who contribute work to receive no reward in a given cycle due to the lottery outcome. If desired to encourage participation, a small percentage of every transaction fee can be reserved and evenly distributed among all or most auditors who participated, ensuring consistent baseline compensation however small. Rewards according to the protocol can remain flexible.

AUDIT BLOCKS

Audit blocks form a compact ordered canonical history of all valid blocks **(BY REFERENCE)** within a given period. Audit blocks include details such as Account balance, account-input, account-output, Receipt Count, and other global state details. The audit chain represents global

state updates that together compute the canonical state of the network. Their purpose is to anchor consensus, finalize state meta details, and mint rewards.

Audit blocks are in part a reference map, essentially pointers, to validated blocks acting as an optimized efficient compact snapshot of that cycle. Audit blocks include additional meta data that helps describe the state ensuring future operations can leverage historical data and handle rewards. Auditors generate the same audit block given the same data.

SYNC TRIE

The Sync Trie is Viat's optimized deterministic Merkle-trie variant built for fast sync, low-cost updates, and efficient consensus. Instead of hashing every step of the path, original hashes are stored in buckets at the ends of paths, and only the necessary branches above them are rehashed. Single-hash buckets bubble their hash upward until they meet branches with siblings, naturally compressing paths and reducing computation. The trie's depth and fanout are tuned to avoid the bloat of traditional Merkle tries, keeping memory usage low and allowing the structure to stay in RAM or be stored efficiently in CBOR. Because path selection is deterministic, nodes holding the same hashes always produce the same trie, and missing data can be quickly pinpointed and shared by bucket. The trie has a diamond-like shape: narrow start, wide middle, narrow end. This shape improves hash caching and supports large hash sets without heavy overhead. This Sync Trie design is ideal for Viat's audit-block model, where the trie remains hot in memory while transaction data moves to tiered storage.

ROLE BASED CONSENSUS MODEL

INTRO

Viat employs a dual-role consensus architecture that maps directly onto two complementary layers of the network. The first layer consists of Arbiters, a high-throughput validation fabric responsible for realtime filtering, correctness checks, and probabilistic finality. The second layer consists of Auditors, the canonical ordering layer that assembles validated data into audit blocks, advances the STATE_ID, and preserves the authoritative historical record.

This separation ensures fast, censorship-resistant validation at the edges while preserving strict, deterministic ordering at the core. Arbiters emphasize speed. Auditors emphasize correctness. Together they create a resilient and quantum-ready consensus model capable of massive parallelism.

Miners avoid constant per-vote signatures by using DIS identities, (UW domain certificates), and UDSP’s authentication-via-encryption handshake. Votes are transmitted over authenticated connections that can be resumed with 0-RTT-style recovery if disrupted. Programmable connection identifiers (compact encodings of KEM/KE public keys) allow miners to recognize and persist other trusted miner connections. This enables trustless voting without per-vote signatures while maintaining strong cryptographic guarantees. The result is a design that remains efficient under legacy cryptography and becomes essential in post-quantum environments where signature and public-key sizes are prohibitively large.

Signatures are sparse and purpose-bound. Rather than signing every vote, miners sign only compact commitments, such as hashes of Sync Trie roots or transactions selected by the deterministic hash-distance lottery. This keeps cryptographic work minimal and focused, while still providing provable accountability.

Probabilistic finality occurs during arbitration; auditors later record the canonical history.

- Conceptually, arbitration provides fast finality while auditing provides slow-time correctness:
  Chamber A (Arbiters): real-time validation and finality.
  Chamber B (Auditors): canonical history, rewards, slashing, and correctness guarantees.

In summary, arbiters form a decentralized finality engine optimized for speed and scale, while auditors form a verification-and-record engine optimized for correctness and durability. This separation is fundamental to achieving high throughput today and survivability in a post-quantum future.

ARBITERS - REALTIME VALIDATION LAYER

Arbiters act as a decentralized, high-speed validation fabric. They verify transaction structure, validate transactions, enforce protocol rules, check balances, formulate finality, and validate signatures. Any node performing this work must present an active stake; un-staked arbiter blocks are rejected immediately.

This arbiter-first design maximizes liveness, censorship resistance, fault tolerance, and throughput. Lightweight arbiters are harder to censor than a smaller auditor set, enabling clients to route transactions through diverse paths even in hostile regions.

Validation load is distributed across the arbiter pool, and arbiters do not need full global state. Only the information required to validate a specific transaction is required keeping state as small and memory-friendly as possible.

AUDITORS - HISTORY, STATE, AWARDS

Auditors do not reverify every transaction. They intervene only when arbiters disagree, submit conflicting hashes, or explicitly flag a trans-

action as invalid. Audits are therefore triggered by inconsistency, not by default. This sharply reduces auditor workload and allows auditors to focus on their primary role: macro-level canonical ordering, state progression, and historical recording.

QUORUM

Arbiter votes are ephemeral. Votes are retained only for a bounded time window and quantity sufficient to reach quorum. Once auditors agree on outcomes, individual arbiter votes are pruned. Audit blocks record only reward receipts and final state commitments, not the full vote history. The audit block itself serves as proof of quorum for that cycle.

If enough authenticated votes are received via auth-via-encryption, consensus can be reached without requiring signatures. If signatures are required, a single signature over a Sync Trie root can attest to many validations at once.

SLASHING & RANDOM SAMPLING

Auditors can request signatures from peers attesting to a set of votes. Fraudulent attestations become self-incriminating proofs and trigger slashing. Randomized signature challenges ensure participation remains unpredictable and resistant to coordination. Miners submitting bad signatures will result in invalidating all votes made by that Arbiter.

Random truth sampling enforces honesty. Auditors may randomly request proof-of-verification for transactions an arbiter claims to have validated, or request validation of randomly selected transactions. Failure to respond or provide valid proof invalidates the arbiter’s votes and can trigger slashing. This keeps the validation pool clean without requiring exhaustive checks.

SYBIL PROTECTIONS

Role-based consensus enforces authorization during connection establishment, mitigating spam and denial-of-service attempts before validation begins. Auditors may operate in isolated, whitelist-based modes, while arbiters remain broadly reachable.

STATE MANAGEMENT

Global state management gets down to auditors once a deterministic amount of transactions occur a soft state change announcement goes out this ensures the network is in sync by referencing the correct state IDs. Any attempt to be out of state automatically invalidates the block.

Transactions reference a future STATE_ID, enabling auditors to reject stale transactions immediately and define a clear right-or-wrong audit outcome. If a wallet sends a new TX block with an incorrect state reference to an arbiter it will be denied.

STATE ANCHORING

All blocks reference the previous and an upcoming audit block ID to account for propagation, lag, and ensure inclusion in upcoming audit cycles.

Blocks reference the last prior known audit block and the future audit block. This helps to account for lag, assist in congestion control, state anchoring, and network sync. The future audit block will reference it the block hash. while the current forming audit block references the prior cycles blocks. This defers canonical history while arbiters handle realtime validation keeping the entire process asynchronous. Auditors include block hashes after sufficient arbiter validation.

FILESYSTEM (WIP)

mempool→cache→DB→filesystem→long-term storage

VIAT features a self-organizing, hierarchical superstructure stored in-part as a (physical and or virtual) filesystem. The cryptographic superstructure integrates both linear and DAG-based blockchains which is reflected in the filesystem. This design enables deterministic structure, spacial-locality, asynchronous operations, constant-time lookups, and high-throughput parallel processing for a post-quantum world.

Each block maps to a deterministic, computable storage path derived from its cryptographic material and metadata. This enables stateless filesystem construction and universal, direct lookup without auxiliary indexes. As blocks are persisted, the filesystem incrementally self-assembles, increasing both physical and logical interconnection over time. The storage path hierarchy is intentionally isomorphic to the blockchain superstructure, ensuring structural alignment between on-disk layout and on-chain topology.

BLOCK PATHS

A block can be referenced and located via a deterministic path generated by the blocks cryptographic details. Block storage is designed with spacial locality and sharing in mind to ensure both local and remote distributed storage is natively supported. Block paths are specifically built for B-Tree based filesystems, modern SSDs, and the latest filesystems.

The block path can vary between full nodes which will have customized VIAT filesystem variants specific to their filesystem, hardware, and config. The standard string based path generation uses HEX encoding and a custom lowercase Base38 encoding consisting of a lowercase variant of Base64URLSafe for compact final folders. This default config is chosen for fast lookups on filesystems with B-Trees or B+Trees, (highly optimized for SSDs), targeting the standard 4KB block size.

LOCAL STRING Block Example:
/wallets/aa/bb/cc/o1mjtm01w8u968f_/WALLET_BLOCK

The filesystem shares some aspects with existing solutions such as Git, Content-Addressable file storage, Object-storage, and efforts to optimize Blockchain Data Retrieval https://data.coindesk.com/blogs/on-chain-series-iv-optimising-blockchain-data-retrieval-with-nfs.

Filesystem exploration
To explore the filesystem a node uses audit blocks which act as a reference map, checkpoint indexes, and or a database which is holding path & indexing info/meta data of recent blocks.

NETWORKING (WIP)

We have reduced this section to focus on Viat related components.

VIAT utilizes two core components of the Universal Web: DIS for domain resolution and UDSP for network communication. Together, they accelerate both transaction processing and node-to-node messaging. The combined DIS+UDSP stack (UW://) provides an exceptionally packet-efficient hybrid post-quantum handshake, enabling the fastest post-quantum communication and transaction requests within the VIAT network. Both components are designed to be viable improved replacements for HTTP and DNS.

UNIVERSAL DATA STREAM PROTOCOL

For network communication, VIAT uses a custom UDP-based protocol called UDSP. UDSP is stateful, encrypted by default, and provides perfect-forward-secrecy while supporting efficient post-quantum handshakes. It delivers out-of-order reliability, bidirectional streams, multiplexing, and authentication-via-encryption. The protocol is designed to pair closely with the DIS, minimize connection setup time, and provide fast, secure, and efficient node-to-node communication across the VIAT network. UDSP is designed to do more with less but in as few packets below typical MTU as possible.

UDSP enforces efficient data practices by using native CBOR encoding rather than text-based formats like JSON, which inflate size when representing binary values. This shift immediately cuts client transaction request sizes by at least 25%, with reductions reaching roughly 33% in typical cases. The benefit is most significant for light clients, block explorers, and wallet applications: from the moment a request leaves the client to the moment it reaches an Auditor, all data remains in a compact, binary representation. This reduces bandwidth, speeds up verification, and ensures consistently lean communication across the network.

DOMAIN INFORMATION SYSTEM

The combination of DNS, Domains, & CAs carry well-known structural weaknesses: it relies on brittle hierarchical authorities, lacks inherent cryptographic identity, returns none-cryptographic domain records, offers no built-in post-quantum security, not designed for UTF-8 native domains, is text based, lacks future-proofing, lacks domain certificate history, and was never designed for authentication-via-encryption. The Universal Web is designed to provide a full solution that combines centralized and decentralized components to address the true technical limitations of DNS.

Most Web3 domain systems do little more than sell domains and place simplistic domain records on the blockchain. This approach does not address the deeper technical problems of the entire Domain System itself, is not society friendly, nor does it prepare the domain layer for a post-quantum environment. Current Web3 Domain Systems don’t address technical flaws, introduces new ones, lack adoption, and in practice still rely on these exact systems to carry out supposed decentralized DNS.

For a comprehensive alternative to exist it must incorporate these aspects: Domain Name format, native encoding support, Domain Name Resolution, Domain Certificates, Domain authority for societal integration, and a Network Data Transport protocol all focused on the traditional web model. Mass adoption & integration hasn’t occurred for Web3 domains due to leaving these aspects unaddressed and features lacking.

For Domain resolution Viat has the Domain Information System which returns fully isolated cryptographic certificates instead of just domain records. Each DIS certificate is self-contained and signed by the publicly verifiable domain owner. This ensures any attempt at altering the contents of the domain certificate will automatically cause signature verification to fail. This offers greater certainty, a bad acting DNS server can’t alter certificates without the direct participation of the owner. In this context a CA is replaced in part by a publicly verifiable per-domain blockchain ledger. However, the UW acts as the domain seller

therefore carries out the initial sale and recurring domain billing cycle akin to the domain model on the Web today.

This model ensures DNS servers can’t provide false records and rogue CAs can’t issue a new certificate for a domain. Only the domain owner can issue a domain certificate first and any involved CA-like entity merely acts as additional verification. This shifts the security model to focus on domain ownership which on chain is transparently verifiable. For Viat’s Arbiter-Auditor model this is vital for securing communications to nodes, enabling fast discovery, eliminate DNS related attacks, protect clients from fraudulent nodes, eliminates CA authorities issuing false certificates, and as fast as possible connection establishment.

AUTHENTICATION-VIA-ENCRYPTION

Authentication-via-encryption is vital for reducing connection establishment time in authentication based blockchain networks. Currently blockchains utilize a typical domain certificate and carry out a signature verification process akin to the Web today via TLS to authenticate a node. This is slow introduces more packets than needed and slows down communication between nodes. Nodes on Viat have a UW domain and therefore have an associated DIS certificate. This DIS Certificate is mandatory self-signed by the signature key pair and contains the key-exchange key-pair. This implicitly associates the two key pairs and allows nodes to use the key-exchange key pair instead of the signature key pair for authentication-via-encryption.

Authentication-via-encryption folds identity verification (authentication) directly into the connection encryption process, reducing round trips and accelerating connection setup. This efficiency is especially important in a post-quantum environment, where public keys and signatures are far larger. By proving possession of the private key through the encryption step itself, the system avoids a separate certificate-signature verification phase and streamlines the entire handshake. It also strengthens the security of the connection. Early packet encryption

can begin as the handshake progresses, and session keys naturally ratchet forward.

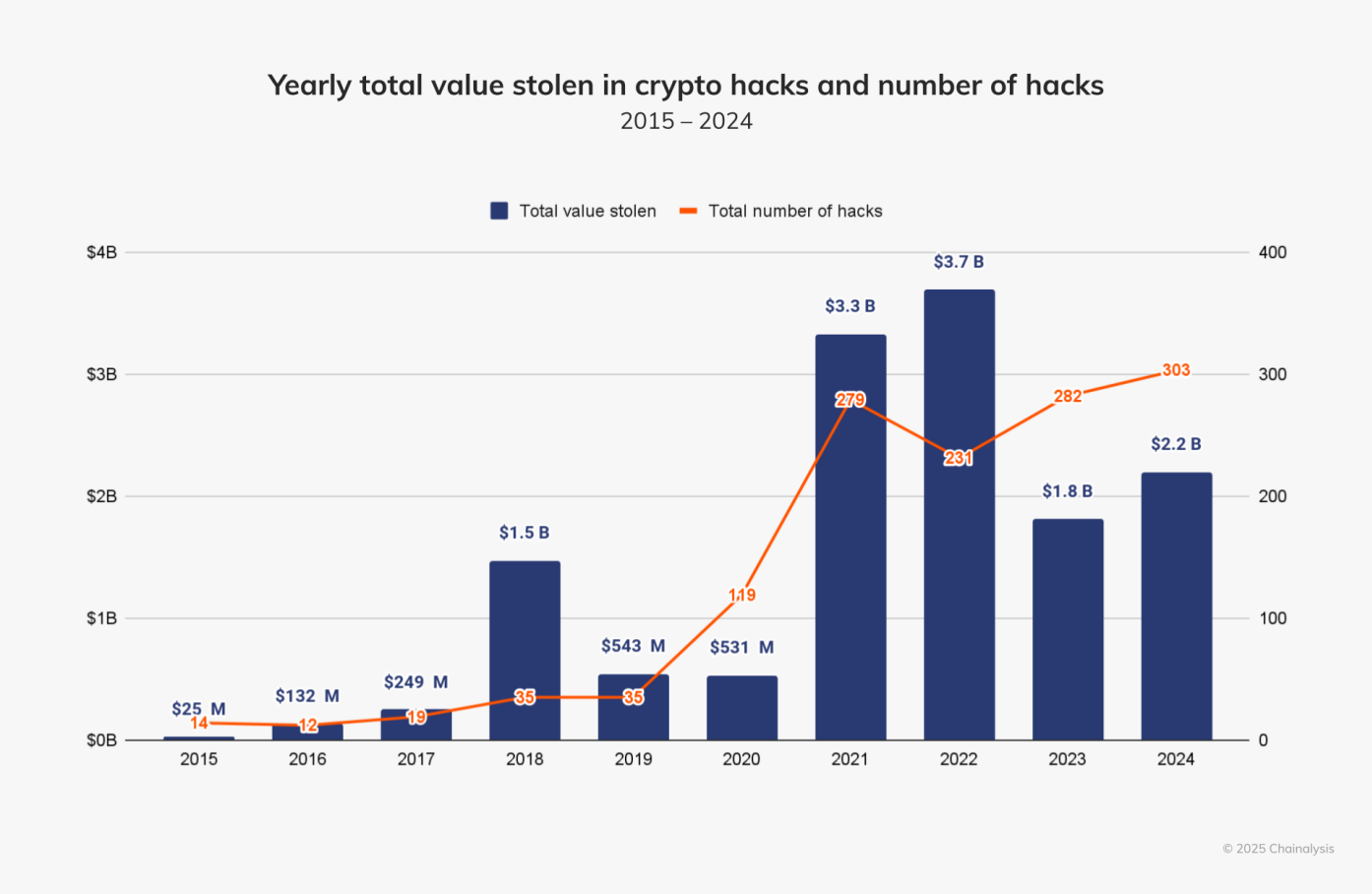
This has been well studied but has faced limitations due to the design of DNS and TLS. The post-quantum variant has been dubbed TLSKEM which cloudflare has experimented with. This feature matters significantly more in the context of a post-quantum era because signatures are significantly larger and so are their domain certificates.

- https://thomwiggers.nl/publication/kemtls/kemtls.pdf
- https://blog.cloudflare.com/kemtls-post-quantum-tls-without-signatures/

Authentication-via-encryption is a vital component for post-quantum cryptocurrencies but to properly take advantage of it a new Domain System, Dual Certificate design, and Network Protocol is required. All of which the UW is designed to meet and Viat fully utilizes.

DNS & WEB EXPLOIT COSTS

- Over a third of all cyber attacks are deployed via DNS. (Global Cyber Alliance, 2021)
- In 2020, cybercrime losses totaled $1 trillion worldwide—1% of the global GDP. (TechRadar, 2021)
- Critical flaws like CVE-2025-40778 in BIND 9 DNS software enable attackers to inject forged records, bypassing security measures and redirecting traffic to phishing sites.
- Recent $16.2M breaches at Garden Finance and others highlight systemic weaknesses, demanding cultural shifts to treat off-chain infrastructure with on-chain security rigor.
- https://www.ainvest.com/news/repeated-dns-hijack-vulnerability-defi-call-chain-security-reforms-2511/
- https://indd.adobe.com/view/5e9b72da-9c31-4aa5-ba4f-960f8db2e881



TOKENOMICS (WIP)

RECYCLING

Recycling in VIAT acts as a real-time controllable deflationary component that seeks to extend the total mining period by indefinitely ensuring there is always block rewards. Unlike BTC VIAT strives to delay reaching halving periods and extend the total amount of time required to mine all VIAT. By constantly recycling VIAT back into the network circulating supply can be lowered and act as a deflationary price control mechanism.

Controlled recycling allows Viat to more aggressively recover from initial bootstrapping used to encourage initial network growth.

DEVELOPMENT

Current prototyping and development is primarily in Node/Bun with some native modules in C++, C, and Zig. However, we plan on moving most if not all of the codebase over to Zig while still supporting native module integration into Node/Bun. Providing native modules written in Zig for Node/Bun is critical to mass-adoption and to reduce the learning curve initially required to build apps or contribute. Most developers understand Web development and providing an environment they are familiar with will be the best possible case for mass adoption. Forcing developers or creators to learn or use low-level languages increases the learning curve and will slow or deter adoption. Higher-level languages like JS (Node/Bun) allow fast prototyping, easy app development, ensure the highest mass adoption potential in the long-term, and still allow native module integration for when performance matters.

UNIVERSAL WEB A NEXUS

The UW is a massive undertaking designed to be a viable replacement for the modern World Wide Web and Web3 in their entirety resembling a new type of system which we call a “nexus”. It’s not an upgrade, merger, or an additional layer on top of the Web (akin to what Web3 did) it resembles something else entirely. It includes components that operate in similar roles such as the DIS compared to DNS but are fundamentally different. Web3 failed to address any of the technical shortcomings of the current WWW and instead was merely an additional layer on top of the existing Web. The UW is a fundamental re-imagining of the Web while also merging it with a cryptocurrency this creates what we currently call a nexus. A nexus incorporates features that exist on the Web today and which are expected in cryptocurrencies but natively merges the two to create a singular system from networks with opposite topological designs.

Web3 largely failed to address, and in many cases outright ignored, the fundamental technical constraints of the modern Web. The result was a user experience that was objectively worse than the systems it aimed to replace. The Universal Web (UW) is designed to exceed the long-term potential of the World Wide Web while remaining grounded in the proven client-server model that enables speed, usability, and operational efficiency.

Blindly applying decentralized blockchains to every function of the Web is a category error. Not all problems benefit from global consensus, permanent state, or trustless execution. For most interactive services, forcing computation, storage, and coordination onto a blockchain guarantees lower performance, higher latency, reduced flexibility, and significantly higher costs. Physics always wins: additional consensus layers, replication, and cryptographic overhead inevitably increase bandwidth usage, computational load, and response times, with those costs ultimately borne by end users.

The existing economic model of the Web remains highly effective. It allows businesses to optimize infrastructure, iterate rapidly, absorb operational costs, and deliver fast, low-friction experiences at scale. This is precisely why the Web achieved global adoption. Recasting conventional Web applications as smart contracts inverts these advantages: optimization becomes constrained, upgrades become costly and risky, and execution fees are pushed directly onto users, degrading both usability and accessibility.

The immutability of smart contracts is poorly suited to long-lived applications and institutional systems, where security assumptions inevitably change. Once logic or cryptography is permanently embedded, broken primitives cannot be repaired without abandoning the original contract or wallet. This is already evident in legacy blockchains, where wallets have no automatic path to transition to post-quantum security while preserving the address, turning immutability into a systemic liability rather than a strength. Hence why Viat is forced to directly address this concern within wallet generation via a post-quantum commitment scheme called the trapdoor.

UW recognizes that decentralization is a powerful tool, but only when applied selectively and intentionally. By preserving the client-server model where it excels, and introducing cryptographic verification, coordination, and decentralization only where they provide real, measurable benefits, the UW avoids the performance, cost, and complexity traps that stalled Web3. The goal is not ideological decentralization, but a system that actually scales, competes, and improves the Web as people use it today without asking them to accept worse tradeoffs in exchange for abstractions they don’t need.

Another document will instead be used to cover the Universal Web.

COPYRIGHT © 2025 TM, UW, VIAT
IN MEMORY OF AMY (1993 - 2025)